Docker & Kubernetes: Beginner to Production Guide

This guide explains Docker and Kubernetes in simple English with real-world production examples. It covers containers, images, networking, orchestration, deployments, scaling, CI/CD, monitoring, security, and cloud deployments.

1. What is Docker?

Docker is a containerization platform.

Containers package:
• Application code
• Dependencies
• Libraries
• Runtime

This allows applications to run consistently everywhere.

Example:
Your app works on your laptop but fails on server due to different environment.
Docker solves this issue.

2. Why Docker is Popular

Benefits:
• Lightweight
• Fast startup
• Portable
• Easy deployment
• Consistent environments

Production Example:
A company runs:
• Backend service
• Frontend app
• Database
• Redis cache

Docker packages each service separately.

3. Containers vs Virtual Machines

Virtual Machines:
• Heavy
• Require full OS

Containers:

• Lightweight
• Share host OS
• Faster startup

Docker containers are more efficient for microservices.

4. Docker Architecture

Main Components:
• Docker Client
• Docker Daemon
• Docker Images
• Docker Containers
• Docker Registry

Flow:
Developer → Docker Build → Image → Container

5. Docker Images

Images are templates used to create containers.

Example:
Python Image
Node.js Image
MySQL Image

Images contain:
• Application code
• Dependencies
• Runtime

6. Docker Containers

Containers are running instances of images.

Example:
Image = Blueprint
Container = Running Application

You can run multiple containers from one image.

7. Dockerfile Explained

Dockerfile defines how image is built.

Example:

```
FROM python:3.11
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["python", "app.py"]
```

This builds a Python application image.

8. Important Docker Commands

docker build
docker run
docker ps
docker stop
docker logs
docker images
docker exec

These commands are used daily in production.

9. Docker Networking

Containers communicate using Docker networks.

Types:
• Bridge Network
• Host Network
• Overlay Network

Production Example:
Frontend container talks to backend container securely.

10. Docker Volumes

Volumes store persistent data.

Without volumes:
Data disappears after container removal.

Example:
MySQL database stores data inside volume.

11. Docker Compose

Docker Compose manages multi-container applications.

Example:
App with:
• Frontend
• Backend
• Database
• Redis

All services start using one command.

12. Real World Docker Example

E-commerce Project:

Containers:
• React frontend
• Spring Boot backend
• MySQL database
• Redis cache
• Nginx reverse proxy

Each service runs independently.

13. Docker in CI/CD

CI/CD pipelines use Docker heavily.

Flow:
1. Developer pushes code
2. CI pipeline builds Docker image
3. Tests run
4. Image deployed to server

Tools:
• GitHub Actions
• Jenkins
• GitLab CI

14. Docker Best Practices

Best Practices:
• Use small images
• Avoid running as root
• Use multi-stage builds
• Keep images lightweight
• Scan for vulnerabilities

15. What is Kubernetes?

Kubernetes is a container orchestration platform.

It manages:
• Deployment
• Scaling
• Networking
• Monitoring
• Failures

Kubernetes automates container management.

16. Why Kubernetes is Needed

Docker runs containers.

But managing thousands of containers manually is difficult.

Kubernetes solves:
• Auto scaling
• Self healing
• Load balancing
• Rolling updates

17. Kubernetes Architecture

Main Components:

Master Node:
• API Server
• Scheduler
• Controller Manager
• etcd

Worker Node:
• Kubelet
• Kube Proxy
• Pods

18. What are Pods?

Pod is the smallest Kubernetes unit.

A pod contains:
• One or more containers

Example:
Backend container + logging sidecar container.

19. Deployments

Deployments manage application replicas.

Example:
You deploy 5 replicas of backend app.

If one pod crashes:
Kubernetes automatically creates new pod.

20. Services in Kubernetes

Services expose applications.

Types:
• ClusterIP
• NodePort
• LoadBalancer

Example:
Frontend accesses backend through Kubernetes Service.

21. ConfigMaps and Secrets

ConfigMaps:
Store configuration.

Secrets:
Store sensitive data like passwords and API keys.

Production systems never hardcode secrets.

22. Auto Scaling

Kubernetes automatically scales applications.

Example:
Traffic increases during sale.
Kubernetes increases pod count automatically.

23. Rolling Updates

Kubernetes updates applications without downtime.

Old pods replaced gradually with new pods.

Users continue using application during deployment.

24. Kubernetes Networking

Every pod gets its own IP.

Kubernetes networking allows:
• Pod communication
• Service discovery
• Load balancing

25. Monitoring Kubernetes

Monitoring Tools:
• Prometheus
• Grafana
• ELK Stack

Important Metrics:
• CPU usage
• Memory usage
• Pod health
• Network traffic

26. Kubernetes Security

Security Features:
• RBAC
• Network Policies
• Secrets
• Pod Security Policies

Production clusters require strong security.

27. Kubernetes in Cloud

Managed Kubernetes Services:
• AWS EKS
• Azure AKS
• Google GKE

Benefits:
• Easy scaling
• Managed infrastructure
• Automatic updates

28. Real Production Example

Food Delivery App:

Services:
• Order Service
• Payment Service
• Notification Service
• Tracking Service

Kubernetes manages all microservices automatically.

29. Docker vs Kubernetes

Docker:
Container platform.

Kubernetes:
Container orchestration platform.

Docker creates containers.
Kubernetes manages containers at scale.

30. Docker + Kubernetes Workflow

Step 1:
Build Docker image.

Step 2:
Push image to Docker Hub.

Step 3:
Deploy image using Kubernetes manifests.

Step 4:
Kubernetes manages scaling and failures.

31. Interview Questions

1. Difference between Docker and VM?
2. What is a container?
3. What is Kubernetes?
4. What is a pod?
5. Explain deployment and service.
6. What is auto scaling?
7. Explain rolling updates.

32. Learning Roadmap

Stage 1:
Learn Docker basics.

Stage 2:
Build Dockerized projects.

Stage 3:
Learn Kubernetes architecture.

Stage 4:
Deploy apps on Kubernetes.

Stage 5:
Learn monitoring and CI/CD.

Stage 6:
Deploy on cloud.

33. Final Advice

Docker and Kubernetes are core technologies in DevOps and cloud engineering.

To master them:
• Build projects
• Practice deployments
• Learn debugging
• Understand networking
• Learn CI/CD deeply

These skills are highly valuable in software industry.

Docker vs Kubernetes Comparison

Feature	Docker	Kubernetes
Purpose	Containerization	Container Orchestration
Scaling	Manual	Automatic
Self Healing	No	Yes
Load Balancing	Limited	Built-in
Complexity	Easy	Advanced

End of Guide